Field Hours

CSCE 713 Software Security

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A diagram before a patch

You start with a data-flow diagram, not with a pile of bug names. Draw each process, each store, and each trust boundary the bytes cross. A small web app often has 3 boundaries: browser to server, server to database, and server to an admin host. At every boundary you ask what can be spoofed, changed, denied, leaked, flooded, or used to gain a role. You write 1 threat per boundary before you pick a control.

You name the asset on the far side of the boundary in the same sentence as the threat. A login form that accepts a string is not yet a threat. The threat is that the string crosses into a query, a file path, or a role check. You give each threat an identifier, such as T2, so a later test can point back at it. A control that traces to no boundary is a guess.

From diagram to one threat

1. Mark trust boundaries

2. Name the asset

3. Write one threat

4. Point a control at it

Three boundaries, one identifier

1. You draw the browser, the server, and the database. That is 3 boundaries once you also include the admin host.

2. On the server-to-database boundary you write T2: an untrusted string must not change the query text.

3. The control you name is a parameter channel. T2 is the identifier the test will cite. You do not start by renaming functions.

Problem 1. You listed 12 possible bugs and never drew a boundary. What is missing?

Answer. The data-flow diagram and a threat tied to a boundary. A bug list without a boundary does not say which asset the bytes can reach. Write 1 threat per boundary, with an identifier, before you pick controls.

Problem 2. Threat T2 says an untrusted string must not change query text. Which boundary is that, in the small app?

Answer. The server-to-database boundary. The browser-to-server boundary is where the string arrives, and T2 is about the string becoming query text on the way into the database.

Lecture 2 · 90 minutes

Three defenses around memory

You treat a memory safety bug as a class you contain, not as a single bad copy. Three mechanisms sit on a typical process. A stack canary is a value, often 8 bytes, placed before the return data and checked before the function returns. If those bytes changed, you abort. Address space layout randomization slides bases so a location that was right in one run is wrong in the next. A page may be writable or executable, not both.

You still remove the bad copy. The 3 mechanisms raise the cost of turning a bad write into control of the process, and they do not make an 8-byte overflow acceptable. You turn them on in the build you ship, and you turn on the same checks in the build you test. A crash that only appears with the canary disabled is a finding you file. It is not a reason to ship the weaker build.

Three contains on one process

Canary checked on return

Bases slide each run

Write or execute, not both

An eight-byte buffer and its canary

1. A function has an 8-byte buffer. The canary sits after it and is checked when the function returns.

2. A write of 9 bytes changes a canary byte. The check fails and the process aborts instead of returning.

3. You file the abort against the function. You do not turn the canary off to make the test look quiet.

Problem 1. The canary is 8 bytes and a write touches 9 bytes of the buffer area. What should the return check do?

Answer. It should fail and abort the process. The canary is there so a write past the 8-byte buffer is noticed before the return. Disabling the check is not a fix.

Problem 2. ASLR made one hardcoded location fail on the second run. Do you still fix the overflow?

Answer. Yes. ASLR is 1 of the 3 defenses. A slide that changes each run is not a repair of the copy that writes past the buffer. You fix the copy and you keep the slide, the canary, and the page rule.

Lecture 3 · 90 minutes

Keep data out of the instruction channel

You assume every byte that crossed a trust boundary is data, not instructions. The mechanism is to pass structure and value in separate channels. A database call sends the query text with a placeholder and sends the value beside it, so a 20-character name cannot close the statement and open another. The same split applies to a command, a file path, and a document that would otherwise mix markup with text.

You check the shape you actually accept. If an account number is 8 digits, you reject a string of length 9 before it reaches the store. Allow-lists are smaller than deny-lists: you name the characters you keep, and you drop the

rest. Encoding on the way out is a second mechanism, for a page that must show the value. Validation on the way in does not replace encoding on the way out. You do both.

Two ways to pass a name

Mixed into text

Value sits inside the query

Length can change the parse

Reject this shape

Separate channels

Placeholder stays fixed

Value rides beside it

Parser sees one statement

An eight-digit account number

1. The field must be 8 digits. The submitted string has length 9, so you reject it at the boundary and you never open a query.

2. A valid value, 8 digits, goes out on the parameter channel. The query text still has one placeholder.

3. When you render that value on a page, you encode it for the page. The input check does not do that job.

Problem 1. An account field must be 8 digits. The input has length 12. Where do you stop it?

Answer. At the boundary, before any query, command, or path use. Length 12 is not the 8-digit shape you accept. A parameter channel is for values that already passed the shape check.

Problem 2. You bound the value as a parameter and then inserted it into a page with no encoding. What did you skip?

Answer. The output encoding. The parameter channel keeps the value out of the query text. A page is a second channel, and validation on the way in does not encode on the way out.

Lecture 4 · 90 minutes

Privileges that fail closed

You give a component the smallest set of rights that lets it do its job, and you default to deny. A service account that only reads 1 table should not hold a database role that can write every table. When a check fails or a dependency is down, you fail closed: the operation stops. An error page that still returns the record is an open failure. You would rather return nothing than return the row.

You split roles that should not meet. The code that renders a page does not also own the migration credential. An admin action requires a second check, not only a hidden link. You review rights on a schedule, and you remove a right that no call has used. A process running as 1 identity for both public traffic and admin traffic will

eventually use the stronger right on the weaker path.

Tighten one service account

List the calls it makes -> Grant only those rights -> Deny on any failed check -> Remove unused rights

One table, read only

1. The service answers a lookup by reading 1 table. You grant that table and that operation, and nothing else.

2. A check cannot reach the policy store. You fail closed and return no row. You do not fall back to a broader role.

3. A review finds a write right nobody calls. You remove it. The public path and the admin path no longer share one identity.

Problem 1. The policy service times out. Your code returns the row anyway so the page still loads. What is the failure mode?

Answer. It fails open. A failed check must stop the operation. Returning the row because a dependency is down hands out data you did not authorize on that request.

Problem 2. The public site and the admin tool share 1 service account that can write every table. What do you change?

Answer. You split the identities and cut the public account down to the calls it makes. If it only needs to read 1 table, that is the grant. Unused write rights come off.

Lecture 5 · 90 minutes

What a static pass can see

A static pass reads the program without running it and looks for a path from a source to a sink. A source is a place untrusted bytes enter, such as a request field. A sink is a place those bytes must not control, such as a query, a path, or an HTML insertion. If the pass finds 14 paths and you confirm 3 of them, you file those 3. You do not close the other 11 by wishing.

You learn the false alarms of the tool you chose. A sanitizer the tool does not recognize will look like an open path. You mark that function as a sanitizer in the tool's model, or you rewrite the call so the safe channel is obvious. You do not suppress a warning by deleting the check. A 0-warning build means the pass is quiet. It does not mean you finished the diagram from the first lecture.

Triage one static warning

1. Find the source

2. Walk to the sink

3. Confirm or explain it

4. Fix the real path

Fourteen warnings, three confirmed

1. The pass reports 14 source-to-sink paths. You open each one and ask whether the bytes can still reach the sink.

2. You confirm 3. One of them builds a query by joining a request field into the text. You change that call to a parameter.

3. You leave the other 11 visible until you either fix them or record why each is not a path. You do not delete the warnings.

Problem 1. The tool reports 14 paths and you confirm 3. How many remain your problem until you explain them?

Answer. 11. Confirming 3 does not clear the rest. Each of the 11 is either a real path you fix or a warning you can explain, including a sanitizer the tool failed to see.

Problem 2. A build with 0 static warnings still has no data-flow diagram. Are you done?

Answer. No. A quiet pass means the tool's checks are quiet. It does not replace a threat per boundary, and it will miss sinks the model does not know.

Lecture 6 · 90 minutes

Tests that treat the boundary as hostile

You write security tests against the threats you already numbered. Each test names an identifier such as T2, sends an input at the boundary, and states the result you expect. A parser that reads a length and then that many bytes must reject a header whose length is 4 when only 2 bytes follow. The expected result is a clean reject and no partial object. A crash is a failure of the test, not a trophy.

A fuzzer is a generator plus an oracle. It mutates inputs and watches for a failed assertion, a timeout, or an abort. You run it on a build that still has the canary and the page rules turned on. You cap a run, for example 20000 cases, and you file every new abort with the input that reached it. You do not ship a parser that only survives the 12 examples you typed by hand.

One security test, end to end

Cite the threat id -> Send the boundary input -> Expect a clean reject -> File any abort

A length that does not match

1. Threat T4 says a short read must not become an object. You send a header that claims length 4 and then you supply 2 bytes.

2. The parser returns a reject and allocates nothing. That is the expected result, so this case passes.

3. A later generated case aborts inside the same parser. You save that input, keep the canary on, and file the abort against T4.

Problem 1. The header says length 4 and the body has 2 bytes. What is the expected result?

Answer. A clean reject and no partial object. The test is tied to the threat that a short read must not become an object. An abort is a failed test, not a success.

Problem 2. Your suite has 12 hand-written cases, all green, and you have never run a generator. What coverage claim is off?

Answer. You have not exercised the boundary beyond those 12 cases. A generator plus an oracle, on a build with the canary still on, is how you look for aborts your examples missed. Green examples are not that run.

Lecture 7 · 90 minutes

Score only what you confirmed

You score a confirmed issue as likelihood times impact. Both are integers from 1 to 5, and the product is the score you record. An issue you have not confirmed scores 0, even if a tool shouted. You only score an issue that crosses a boundary you drew. After a control, you rescore. If likelihood drops from 4 to 1 and impact stays 5, the score drops from 20 to 5. The 5 is the residual you still own.

You spend time on the highest residual, not on the loudest headline. A score of 20 that you can cut to 5 this week outranks a score of 6 you cannot touch yet. You write the residual next to the threat identifier so the diagram stays honest. Risk acceptance is a sentence a person signs: they accept residual 5 on T2 for 30 days. Silence is not acceptance.

Before and after one control

Before the control

Likelihood 4, impact 5

Product is 20

Boundary crossing confirmed

Residual

Likelihood 1, impact 5

Product is 5

Owner signs the 5

From twenty down to five

1. T2 is confirmed on the database boundary. Likelihood 4, impact 5, score 20.

2. You move the value onto a parameter channel. Likelihood falls to 1. Impact stays 5 because the data is still sensitive. Residual score is 5.

3. A named owner accepts residual 5 for 30 days, in writing. The unsigned tool warning you did not confirm stays at 0.

Problem 1. Likelihood is 3 and impact is 4 on a confirmed boundary crossing. What is the score?

Answer. 12. The score is likelihood times impact, so 3 times 4 is 12. Both factors are on the 1 to 5 scale.

Problem 2. A tool warning is real-looking but you have not confirmed it, and it does not cross a boundary you drew. What score do you record?

Answer. 0. Unconfirmed issues score 0, and you only score crossings of a boundary you drew. Volume of warnings is not a score.

Lecture 8 · 90 minutes

A design you can now review

You can now review a system instead of hunting for a clever bug. You draw the boundaries, write one threat on each, and keep data off the instruction channel. You leave the 3 memory defenses on in the build you ship. You turn a static path into a confirmed issue or a written explanation, and you add a test that cites the threat. You score only confirmed crossings: likelihood times impact, from 1 to 5, and everything else scores 0.

You can also say what you did not finish. A residual score still needs an owner and a date. A green run of 12 hand-written cases is not a fuzz campaign. Practice on one service with 3 boundaries. Confirm the crossings, rescore after each control, and fail closed when a check cannot run. If a right is unused, you remove it. That stack is the review.

The review you can now run

Diagram and threat ids

Defenses and safe channels

Tests that cite threats

Scores with an owner

Two warnings, one score

1. You drew the boundaries. Warning A crosses one of them. Likelihood 2, impact 4, so its score is 8.

2. Warning B does not cross a boundary you drew. Its score is 0 even though a tool ranked it first.

3. You add the products. The review total is 8. You attach a test id to warning A and you leave B explained, not silenced.

Problem 1. Warning A crosses a boundary you drew, with likelihood 2 and impact 5. Warning B does not cross a boundary, with likelihood 5 and impact 5. Using only confirmed boundary crossings, what total do you record?

Answer. 10. A scores 2 times 5, which is 10. B scores 0 because it is not a confirmed crossing of a boundary you drew. The total is 10, not 35.

Problem 2. After that review you still have residual 10, no named owner, and only 12 hand-written tests. What is unfinished?

Answer. Residual 10 needs an owner and a date, and 12 hand-written cases are not a generator run. The score is not acceptance until someone signs it, and the tests should cite the threat identifier.